

Lesson 9 — Polynomial Representation

1. Where We Are in the Curriculum

You have completed the sparse matrix part of the course.

Before Lesson 9, you learned:

Lesson	Main idea
Lesson 1	Why sparse matrices save memory
Lesson 2	Coordinate List representation and creation
Lesson 3	Complete C sparse matrix display program
Lesson 4	Sparse matrix addition algorithm
Lesson 5	Program to add two sparse matrices
Lesson 6	Complete sparse matrix addition in C
Lesson 7	Sparse matrix using C++ classes
Lesson 8	Sparse matrix operator overloading in C++

Lesson 9 starts a new unit: **Polynomial Data Structures**.

In this unit, you will learn how to represent, create, display, evaluate, and add polynomials using data structures.

Lesson 9 focuses only on **representing a polynomial in memory**.

2. Big Idea of Lesson 9

A polynomial is made of terms.

Instead of storing the polynomial as a string like this:

3x⁵ + 2x⁴ + 5x² + 2x + 7

we store each term using two numbers:

coefficient, exponent

So the polynomial becomes a list of coefficient/exponent pairs.

This is similar to sparse matrix storage:

Sparse matrix idea	Polynomial idea
Store only useful non-zero matrix values	Store only actual polynomial terms
One sparse element stores row, column, value	One polynomial term stores coefficient, exponent
Dynamic array stores sparse elements	Dynamic array stores polynomial terms
<code>num</code> means number of non-zero elements	<code>n</code> means number of polynomial terms

The main idea is:

A polynomial can be stored as an array of terms.

3. What Is a Polynomial?

A polynomial is an algebraic expression made by adding terms.

Example:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

This polynomial has 5 terms:

$3x^5$
 $2x^4$
 $5x^2$
 $2x$
 7

Each term has a number part and a power part.

4. What Is a Term?

A **term** is one part of a polynomial.

Example:

$$3x^5$$

This term has two important parts:

Part	Meaning	Value in $3x^5$
Coefficient	The number multiplied by x	3
Exponent	The power of x	5

So:

$$3x^5$$

is stored as:

```
coefficient = 3  
exponent = 5
```

5. Coefficient

The **coefficient** is the number in front of the variable.

Example:

$$5x^2$$

Here:

```
coefficient = 5
```

Another example:

$$-4x^3$$

Here:

$$\text{coefficient} = -4$$

The coefficient tells us how many times the power of x is multiplied.

6. Exponent

The **exponent** is the power of x .

Example:

$$5x^2$$

Here:

$$\text{exponent} = 2$$

Another example:

$$3x^5$$

Here:

$$\text{exponent} = 5$$

The exponent tells us how many times x is multiplied by itself.

For example:

$$x^5 = x \times x \times x \times x \times x$$

7. Important Polynomial Rules

Rule 1: If a coefficient is written, use that coefficient

Example:

$$7x^3$$

This means:

```
coefficient = 7  
exponent = 3
```

Rule 2: If no coefficient is written, the coefficient is 1

Example:

$$x^4$$

This actually means:

$$1x^4$$

So we store it as:

```
coefficient = 1  
exponent = 4
```

Another example:

$$-x^4$$

This means:

$$-1x^4$$

So we store it as:

```
coefficient = -1  
exponent = 4
```

Rule 3: If x is written without an exponent, the exponent is 1

Example:

```
2x
```

This actually means:

```
2x^1
```

So we store it as:

```
coefficient = 2  
exponent = 1
```

Another example:

```
x
```

This means:

```
1x^1
```

So we store it as:

```
coefficient = 1  
exponent = 1
```

Rule 4: If there is no x , the exponent is 0

Example:

7

This actually means:

$7x^0$

Why?

Because:

$x^0 = 1$

So:

$7x^0 = 7 \times 1 = 7$

Therefore, the constant term **7** is stored as:

coefficient = 7
exponent = 0

8. Example: Convert a Polynomial into Terms

Take this polynomial:

$3x^5 + 2x^4 + 5x^2 + 2x + 7$

Break it into terms:

Term	Coefficient	Exponent
$3x^5$	3	5
$2x^4$	2	4
$5x^2$	5	2
$2x$	2	1

Term	Coefficient	Exponent
7	7	0

So the polynomial can be stored as:

```
(3, 5)
(2, 4)
(5, 2)
(2, 1)
(7, 0)
```

Each pair means:

```
(coefficient, exponent)
```

9. What We Store and What We Do Not Store

When storing a polynomial in a program, we do not need to store symbols like:

```
x
+
^
```

We only need to store:

```
coefficient
exponent
```

For example:

```
3x^5
```

is stored as:

```
3 5
```

The program can later use these two values to display, evaluate, or add the polynomial.

10. Why We Store Polynomials This Way

A polynomial is easier to process when each term is stored separately.

For example, if we want to evaluate:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

we need to process each term one by one.

If each term is stored as:

coefficient, exponent

then the program can easily calculate:

$$\text{coefficient} \times x^{\text{exponent}}$$

This will be useful in Lesson 10, where you will evaluate polynomials.

It will also be useful in Lessons 11 and 12, where you will add polynomials.

11. The `term` Structure

One polynomial term needs two values:

coefficient
exponent

So we create a structure called `term`.

```
struct term {  
    int coef;  
    int exp;  
};
```

Meaning of each field:

Field	Meaning
<code>coef</code>	coefficient
<code>exp</code>	exponent

Example:

```
struct term t;
t.coef = 3;
t.exp = 5;
```

This represents:

$3x^5$

One `term` does not store the whole polynomial.

It stores only one part of the polynomial.

12. The `poly` Structure

A polynomial has many terms.

So we need another structure to represent the whole polynomial.

```
struct poly {
    int n;
    struct term *t;
};
```

Meaning of each field:

Field	Meaning
<code>n</code>	number of terms in the polynomial
<code>t</code>	pointer to a dynamic array of terms

Example:

For:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

there are 5 terms.

So:

$$n = 5$$

The pointer `t` will point to an array of 5 `term` records.

13. Important Meaning of `n`

In sparse matrix lessons, `n` often meant:

number of columns

But in polynomial lessons, `n` means:

number of terms

This is very important.

Example:

$$3x^5 + 7$$

This polynomial has 2 terms:

$$\begin{array}{l} 3x^5 \\ 7 \end{array}$$

So:

$$n = 2$$

Even though the highest exponent is 5, `n` is not 5.

Correct:

n = number of terms

Wrong:

n = highest exponent

14. What the Pointer `t` Stores

Inside the `poly` structure, we have:

```
struct term *t;
```

This means `t` is a pointer to an array of terms.

If:

n = 5

then `t` points to an array like this:

```
t[0]  
t[1]  
t[2]  
t[3]  
t[4]
```

Each array position stores one term.

For the example polynomial:

$3x^5 + 2x^4 + 5x^2 + 2x + 7$

we store:

Index	coef	exp	Meaning
0	3	5	$3x^5$
1	2	4	$2x^4$
2	5	2	$5x^2$
3	2	1	$2x$
4	7	0	7

So:

```
t[0] stores the first term
t[1] stores the second term
t[2] stores the third term
...
```

15. Stack Object and Heap Array

In `main`, we may create a polynomial like this:

```
struct poly p;
```

This creates the `poly` object `p` as a normal local variable.

This object usually lives on the stack.

But the term array is created dynamically:

```
p.t = new term[p.n];
```

This array lives on the heap.

Memory picture:

Stack:

```
p
├─ n
└─ t ──────────────────> Heap:
```

```
t[0]  
t[1]  
t[2]  
t[3]  
t[4]
```

Beginner mental model:

```
p is the main polynomial box.  
p.t points to the list of terms.
```

16. Why We Need Dynamic Memory

Before the program runs, we do not know how many terms the user will enter.

The user might enter:

```
3 terms
```

or:

```
5 terms
```

or:

```
20 terms
```

So the program should not use a fixed-size array blindly.

Instead, the program first reads `n`:

```
cin >> p->n;
```

Then it creates exactly enough memory:

```
p->t = new term[p->n];
```

If:

```
p->n = 5
```

then this creates space for:

```
5 term records
```

The array positions are:

```
p->t[0]  
p->t[1]  
p->t[2]  
p->t[3]  
p->t[4]
```

17. Why We Do Not Store the Polynomial as One String

A beginner might think we should store the polynomial like this:

```
string p = "3x^5 + 2x^4 + 5x^2 + 2x + 7";
```

This is possible, but it is not useful for data structure operations.

Why?

Because later we need to:

```
evaluate the polynomial  
add two polynomials  
compare exponents  
combine terms with the same exponent
```

Those tasks are easier when the polynomial is stored as separate terms.

So data-structure representation is better:

```
(3, 5), (2, 4), (5, 2), (2, 1), (7, 0)
```

18. The `create()` Function

The `create()` function builds the polynomial representation from user input.

It reads:

1. Number of terms
2. Coefficient and exponent of each term
3. Stores each term in the dynamic array

A beginner-friendly C++ style version:

```
#include <iostream>
using namespace std;

struct term {
    int coef;
    int exp;
};

struct poly {
    int n;
    struct term *t;
};

void create(struct poly *p) {
    cout << "Enter number of terms: ";
    cin >> p->n;

    p->t = new term[p->n];

    cout << "Enter coefficient and exponent for each term:\n";
    for (int i = 0; i < p->n; i++) {
        cin >> p->t[i].coef >> p->t[i].exp;
    }
}
```

19. Why `create()` Receives a Pointer

The function header is:

```
void create(struct poly *p)
```


This means `p` is a pointer to a `poly` structure.

In `main`, we call:

```
create(&p);
```

The `&` symbol means address.

So:

```
&p
```

means:

```
address of p
```

We pass the address because `create()` must modify the original polynomial object.

It must fill:

```
p.n  
p.t  
p.t[0]  
p.t[1]  
...
```

If we passed only a copy, the original polynomial in `main` would not be properly updated.

20. Why We Use `->` Inside `create()`

Inside this function:

```
void create(struct poly *p)
```

`p` is a pointer.

So we access fields using the arrow operator:

```
p->n  
p->t
```

Wrong inside `create()` :

```
p.n  
p.t
```

Simple rule:

Situation	Operator
You have a normal structure object	<code>.</code>
You have a pointer to a structure	<code>-></code>

Example in `main` :

```
p.t
```

because `p` is a normal object.

Example inside `create()` :

```
p->t
```

because `p` is a pointer.

21. Reading Each Term

Inside `create()` , the loop is:

```
for (int i = 0; i < p->n; i++) {  
    cin >> p->t[i].coef >> p->t[i].exp;  
}
```

If:

```
p->n = 5
```

then the loop runs 5 times.

The variable `i` is the array index.

It starts from 0 because C and C++ arrays are 0-based.

So the input terms are stored like this:

```
First term  → p->t[0]
Second term → p->t[1]
Third term  → p->t[2]
Fourth term → p->t[3]
Fifth term  → p->t[4]
```

22. Full Dry Run

Suppose the polynomial is:

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

The user enters:

```
5
3 5
2 4
5 2
2 1
7 0
```

The first input is:

```
5
```

So:

```
p->n = 5
```

Then the program allocates:

```
p->t = new term[5];
```

Now there is space for 5 terms.

Iteration 1

```
i = 0  
input = 3 5
```

Stored as:

```
p->t[0].coef = 3  
p->t[0].exp = 5
```

Meaning:

$3x^5$

Iteration 2

```
i = 1  
input = 2 4
```

Stored as:

```
p->t[1].coef = 2  
p->t[1].exp = 4
```

Meaning:

$2x^4$

Iteration 3

```
i = 2  
input = 5 2
```

Stored as:

```
p->t[2].coef = 5  
p->t[2].exp = 2
```

Meaning:

$5x^2$

Iteration 4

```
i = 3  
input = 2 1
```

Stored as:

```
p->t[3].coef = 2  
p->t[3].exp = 1
```

Meaning:

$2x$

Iteration 5

```
i = 4  
input = 7 0
```

Stored as:

```
p->t[4].coef = 7  
p->t[4].exp = 0
```

Meaning:

7

Final stored form:

Index	Coefficient	Exponent	Term
0	3	5	$3x^5$
1	2	4	$2x^4$
2	5	2	$5x^2$
3	2	1	$2x$
4	7	0	7

23. Displaying a Polynomial

Lesson 9 mainly focuses on representation, but it is helpful to understand how the stored form can be printed.

A simple display function:

```
void display(struct poly p) {
    for (int i = 0; i < p.n; i++) {
        cout << p.t[i].coef << "x^" << p.t[i].exp;

        if (i != p.n - 1) {
            cout << " + ";
        }
    }
    cout << endl;
}
```

This function receives the polynomial by value:

```
void display(struct poly p)
```

So inside `display()`, `p` is a normal object, not a pointer.

That is why we use:

```
p.n  
p.t[i].coef  
p.t[i].exp
```

not:

```
p->n  
p->t[i].coef  
p->t[i].exp
```

24. Simple Display Output

If the stored polynomial is:

Index	Coefficient	Exponent
0	3	5
1	2	4
2	5	2
3	2	1
4	7	0

The simple display function prints:

```
3x^5 + 2x^4 + 5x^2 + 2x^1 + 7x^0
```

Mathematically, we usually write this as:

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

The program version is more explicit, which is okay for beginners.

25. Complete Beginner Program

```
#include <iostream>  
using namespace std;
```

```

struct term {
    int coef;
    int exp;
};

struct poly {
    int n;
    struct term *t;
};

void create(struct poly *p) {
    cout << "Enter number of terms: ";
    cin >> p->n;

    p->t = new term[p->n];

    cout << "Enter coefficient and exponent for each term:\n";
    for (int i = 0; i < p->n; i++) {
        cin >> p->t[i].coef >> p->t[i].exp;
    }
}

void display(struct poly p) {
    for (int i = 0; i < p.n; i++) {
        cout << p.t[i].coef << "x^" << p.t[i].exp;

        if (i != p.n - 1) {
            cout << " + ";
        }
    }
    cout << endl;
}

int main() {
    struct poly p;

    create(&p);

    cout << "Polynomial is:\n";
    display(p);

    delete[] p.t;

    return 0;
}

```


26. Example Input and Output

Input

```
5
3 5
2 4
5 2
2 1
7 0
```

Stored Meaning

```
3x^5 + 2x^4 + 5x^2 + 2x + 7
```

Output

```
Polynomial is:
3x^5 + 2x^4 + 5x^2 + 2x^1 + 7x^0
```

27. Memory Cleanup

Because the term array is created using:

```
new term[p.n]
```

we should release it using:

```
delete[] p.t;
```

This is important because dynamic memory should be released when it is no longer needed.

Memory rule:

Allocation	Deallocation
<code>new</code>	<code>delete</code>
<code>new[]</code>	<code>delete[]</code>

Allocation	Deallocation
<code>malloc</code>	<code>free</code>

Since we used `new[]`, we use `delete[]`.

28. Order of Terms

For Lesson 9, it is best to store terms in descending order of exponent.

Example of good order:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

Stored as:

```
3 5
2 4
5 2
2 1
7 0
```

The exponents are:

5, 4, 2, 1, 0

This order is useful later when adding polynomials.

In polynomial addition, the program compares exponents and merges terms with the same exponent.

So sorted terms make the later algorithm easier.

29. Complexity Analysis

Let:

`n` = number of polynomial terms

Creation Time Complexity

The `create()` function reads each term once.

If there are `n` terms, the loop runs `n` times.

So:

```
create() time complexity = O(n)
```

Creation Space Complexity

The program creates an array of `n` term records.

So:

```
create() space complexity = O(n)
```

Display Time Complexity

The display function prints each term once.

So:

```
display() time complexity = O(n)
```

Display Extra Space Complexity

The display function only uses a loop variable.

So:

```
display() extra space complexity = O(1)
```

30. Complexity Summary Table

Operation	Time Complexity	Space Complexity	Reason
Create polynomial	<code>O(n)</code>	<code>O(n)</code>	Reads and stores <code>n</code> terms

Operation	Time Complexity	Space Complexity	Reason
Display polynomial	$O(n)$	$O(1)$ extra	Prints each term once
Store polynomial	—	$O(n)$	Dynamic array stores n terms

31. Beginner Mental Model

Think of a polynomial as a row of boxes.

Each box stores one term.

For:

$$3x^5 + 2x^4 + 5x^2 + 2x + 7$$

the boxes are:

```
Box 0: coefficient 3, exponent 5
Box 1: coefficient 2, exponent 4
Box 2: coefficient 5, exponent 2
Box 3: coefficient 2, exponent 1
Box 4: coefficient 7, exponent 0
```

So the program does not need to remember the whole expression as text.

It only remembers the useful numeric information.

32. Connection to Sparse Matrices

This lesson should feel familiar because it uses the same style as sparse matrix storage.

Sparse matrix element:

```
row, column, value
```

Polynomial term:

coefficient, exponent

Sparse matrix structure:

```
struct Sparse {  
    int m;  
    int n;  
    int num;  
    struct Element *e;  
};
```

Polynomial structure:

```
struct poly {  
    int n;  
    struct term *t;  
};
```

Both use:

- a structure for one item
- a structure for the whole object
- a dynamic array to store many items

33. Common Beginner Mistakes

Mistake 1: Thinking `n` means highest exponent

Wrong:

`n` = highest exponent

Correct:

`n` = number of terms

Example:

$$3x^5 + 7$$

This has:

$$n = 2$$

not:

$$n = 5$$

Mistake 2: Forgetting that a constant has exponent 0

Wrong idea:

7 has no exponent

Correct idea:

$$7 = 7x^0$$

So store it as:

$$\begin{aligned} \text{coef} &= 7 \\ \text{exp} &= 0 \end{aligned}$$

Mistake 3: Forgetting that x alone has coefficient 1 and exponent 1

Example:

x

This means:

$$1x^1$$

So store it as:

```
coef = 1
exp = 1
```

Mistake 4: Forgetting that `2x` has exponent 1

Wrong:

```
2x has no exponent
```

Correct:

```
2x = 2x^1
```

So store it as:

```
coef = 2
exp = 1
```

Mistake 5: Storing unnecessary symbols

Wrong idea:

```
Store x, +, and ^ symbols in the array
```

Correct idea:

```
Store only coefficient and exponent
```

The symbols are only needed for display, not for internal storage.

Mistake 6: Using `.` when you should use `->`

Inside:

```
void create(struct poly *p)
```

`p` is a pointer.

Correct:

```
p->n  
p->t
```

Wrong:

```
p.n  
p.t
```

Mistake 7: Forgetting to release dynamic memory

If you allocate with:

```
new term[p.n]
```

release with:

```
delete[] p.t;
```

34. Important Vocabulary

Term	Simple meaning
Polynomial	Expression made of terms
Term	One part of a polynomial
Coefficient	Number multiplied by the variable
Exponent	Power of the variable
Constant term	Term without <code>x</code> , exponent is 0
<code>struct term</code>	Structure that stores one term

Term	Simple meaning
<code>coef</code>	Coefficient field
<code>exp</code>	Exponent field
<code>struct poly</code>	Structure that stores the whole polynomial
<code>n</code>	Number of terms
<code>t</code>	Pointer to dynamic term array
Stack	Memory for normal local variables like <code>struct poly p</code>
Heap	Dynamic memory used for the term array
<code>new[]</code>	Creates a dynamic array in C++
<code>delete[]</code>	Releases a dynamic array created with <code>new[]</code>
<code>-></code>	Access fields through a pointer
<code>.</code>	Access fields through a normal object

35. Quick Self-Check Questions

Question 1

For this term:

$9x^4$

What is the coefficient?

Answer:

9

What is the exponent?

Answer:

4

Question 2

For this term:

x^6

What is the coefficient?

Answer:

1

What is the exponent?

Answer:

6

Question 3

For this term:

8

What is the coefficient?

Answer:

8

What is the exponent?

Answer:

0

Question 4

For this polynomial:

$4x^3 + 2x + 9$

How many terms are there?

Answer:

3

Stored form:

(4, 3)
(2, 1)
(9, 0)

Question 5

If a function receives:

```
struct poly *p
```

should we use `.` or `->`?

Answer:

Use `->` because `p` is a pointer.

36. Final Lesson 9 Summary

Lesson 9 teaches how to represent a polynomial using data structures.

A polynomial is made of terms.

Each term has:

coefficient
exponent

So one term is represented as:

```
struct term {  
    int coef;  
    int exp;  
};
```

The full polynomial is represented as:

```
struct poly {  
    int n;  
    struct term *t;  
};
```

For this polynomial:

$3x^5 + 2x^4 + 5x^2 + 2x + 7$

we store:

(3, 5)
(2, 4)
(5, 2)
(2, 1)
(7, 0)

Important memory idea:

```
struct poly p → stack object  
p.t          → pointer to heap array
```

Important complexity idea:

Creation time = $O(n)$
Creation space = $O(n)$

Most important sentence:

A polynomial is stored as a dynamic array of coefficient/exponent pairs.